

Experiment - 9

Name: K. SRI LASYA

Regno: 192472216

Course Code: MLA0305

Slot: B

Aim

To implement a **Deep Q-Network (DQN)** using TensorFlow and Keras for solving the CartPole reinforcement learning environment.

Algorithm / Procedure

1. Initialize the CartPole environment.
2. Define a neural network to approximate Q-values for each action.
3. Initialize the replay memory and DQN parameters.
4. Select actions using the epsilon-greedy strategy.
5. Store state, action, reward, and next state in memory.
6. Sample experiences from memory and calculate target Q-values.
7. Train the neural network using the calculated Q-values.
8. Reduce the exploration rate after each episode.
9. Record the reward obtained in each episode.
10. Plot the training performance of the DQN agent.

Code

```
!pip -q install gymnasium tensorflow

batch = random.sample(memory, batch_size)

states = np.array([x[0] for x in batch])
actions = np.array([x[1] for x in batch])
rewards = np.array([x[2] for x in batch])
next_states = np.array([x[3] for x in batch])
dones = np.array([x[4] for x in batch])

current_q = model.predict(

    states,

    verbose=0

)
```

```

        next_q = model.predict(
            next_states,
            verbose=0
        )
    for i in range(batch_size):
        if dones[i]:
            target = rewards[i]
        else:
            target = (
                rewards[i]
                + gamma * np.max(next_q[i])
            )
        current_q[i][actions[i]] = target

    model.fit(
        states,
        current_q,
        epochs=1,
        verbose=0
    )
    epsilon = max(
        epsilon_min,
        epsilon * epsilon_decay
    )

    rewards_per_episode.append(total_reward)

    print(
        "Episode:",
        episode + 1,
        "| Reward:",
        int(total_reward)
    )

```

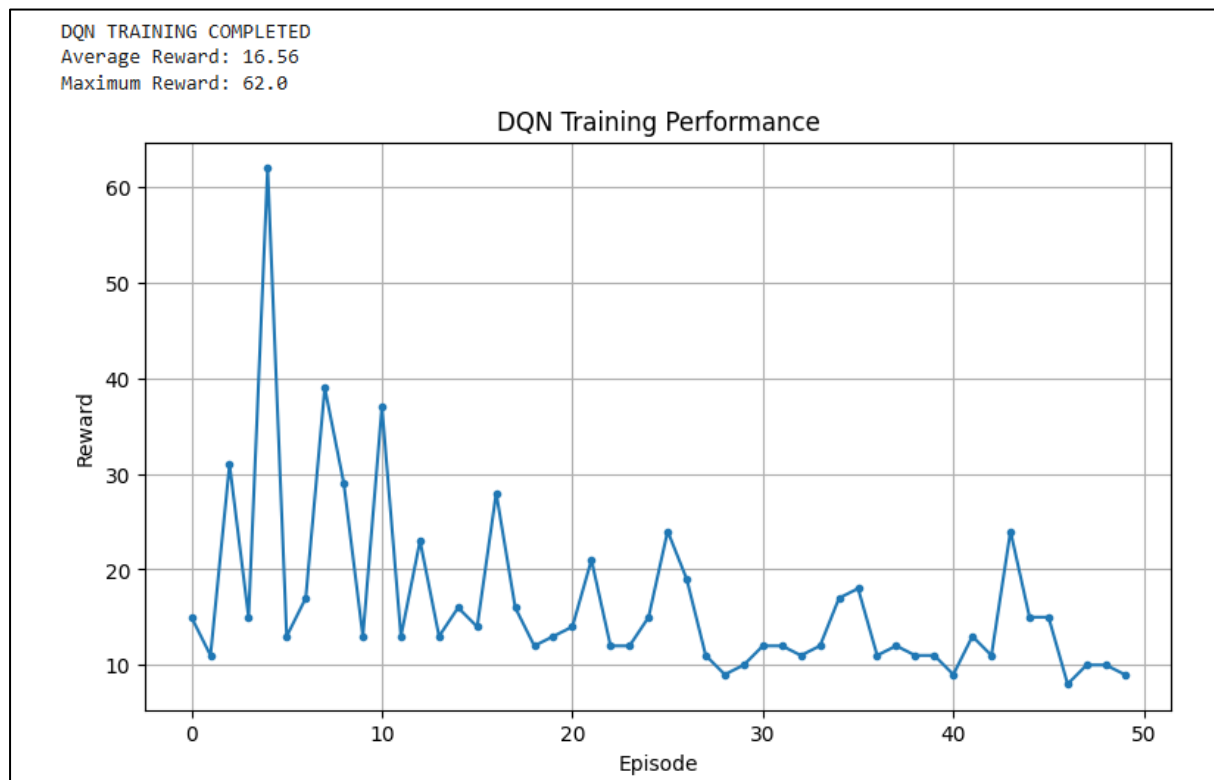
```
)
env.close()

# Results
print("\nDQN TRAINING COMPLETED")

print(
    "Average Reward:",
    round(np.mean(rewards_per_episode), 2)
)
print(
    "Maximum Reward:",
    int(np.max(rewards_per_episode))
)
# Plot graph
plt.figure(figsize=(10, 5))
plt.plot(
    range(1, episodes + 1),
    rewards_per_episode,
    marker="o"
)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("DQN Training Performance")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Output



Result

The Deep Q-Network was successfully implemented using TensorFlow and Keras, and the performance of the agent was evaluated using the rewards obtained during training.